SECURITY OPERATIONS CENTER (SOC) IMPLEMENTATION & VALIDATION REPORT

# Wazuh SIEM Custom Threshold Alerting & Security Operations Dashboard Deployment

End-to-End Configuration, Visualization Engineering, and Empirical Trigger Verification of Custom Rule 100201 (MITRE ATT&CK T1110 – Brute Force) on Ubuntu Linux

## 01 Executive Summary & Lab Objectives

This technical report documents the design, deployment, and empirical verification of a custom Security Operations Center (SOC) monitoring pipeline built on Wazuh SIEM running on Ubuntu Linux (hostname: UBUNTU, management endpoint: https://10.102.20.93). The objective of this implementation was to transition from passive default log ingestion to proactive, threshold-based threat detection paired with real-time analyst visualizations.

Specifically, the deployment achieved three core operational milestones:

- **Custom Dashboard Engineering:** Constructed the 'Ubuntu Security Operations Dashboard' within OpenSearch Dashboards, integrating a time-series line chart tracking authentication failure spikes (Rule 5503) alongside a MITRE ATT&CK technique distribution donut chart.
- **Threshold-Based Correlation Rule (Rule 100201):** Authored and deployed custom XML detection logic in /var/ossec/etc/rules/local_rules.xml to elevate repeated PAM authentication failures (>5 occurrences within a 10 minutes window) into a high-severity Level 10 alert mapped to MITRE ATT&CK technique T1110.
- **Empirical Trigger Validation:** Simulated a rapid brute-force authentication sequence on the Ubuntu host and verified end-to-end telemetry ingestion, correlation, and alert firing within the Wazuh Security Events module.

> **Operational Verification Outcome**
>
> Status: VERIFIED & OPERATIONAL — Rule 100201 triggered at Level 10 on Nov 12, 2026 @ 10:47:38.628 on agent 'UBUNTU' following simulated authentication failures, confirming accurate frequency-window correlation and dashboard visibility.

## 02 SOC Visualization & Dashboard Architecture

To provide security analysts with immediate situational awareness of authentication anomalies and adversarial tactics, a dedicated two-panel dashboard titled 'Ubuntu Security Operations Dashboard' was provisioned via the OpenSearch Dashboards interface.

### 2.1 Dashboard Initialization & Index Pattern Selection

The workflow began by initializing an empty dashboard canvas (Figure 1) and entering Edit mode to create new visualizations bound to the primary Wazuh alert index pattern (wazuh-alerts-*). Selecting wazuh-alerts-* ensures that all panels dynamically query real-time normalized security events generated by the Wazuh analysis engine.

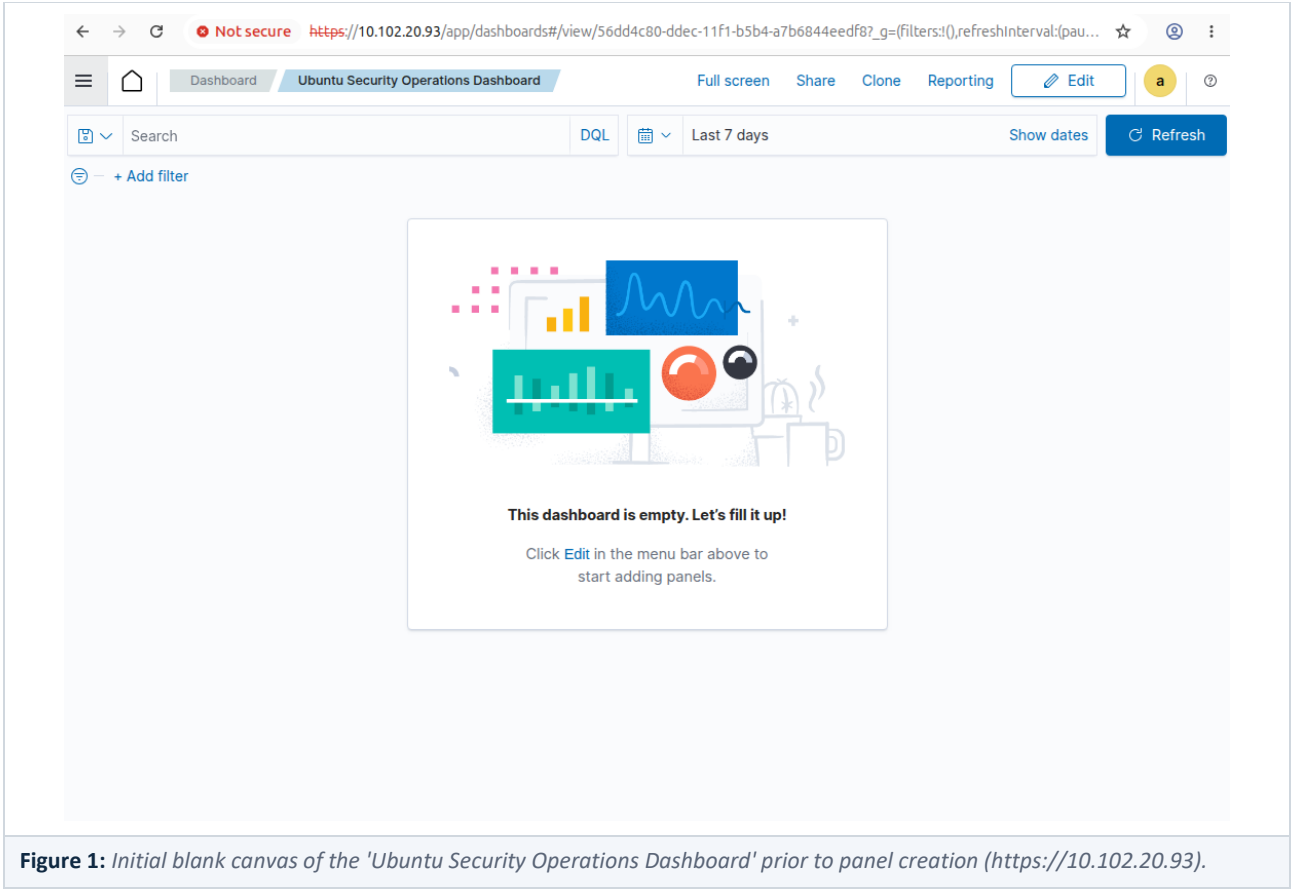



**Figure 1:** *Initial blank canvas of the 'Ubuntu Security Operations Dashboard' prior to panel creation (https://10.102.20.93).*

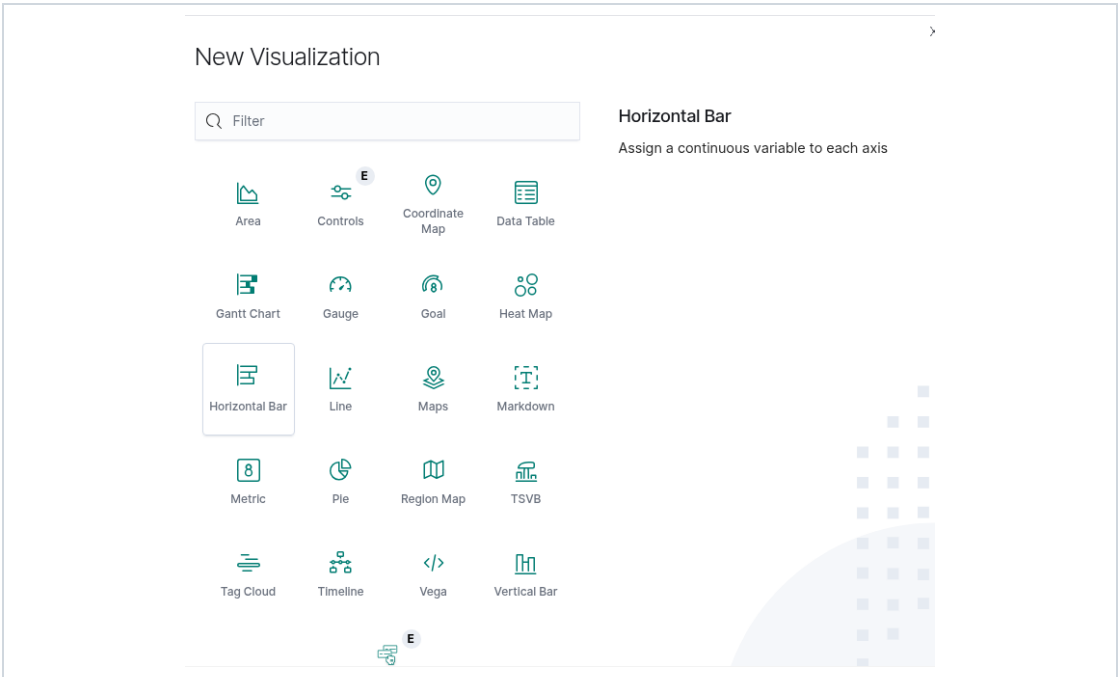



**Figure 2:** *OpenSearch Dashboards visualization type selector used to provision Line and Pie/Donut analytical panels.*

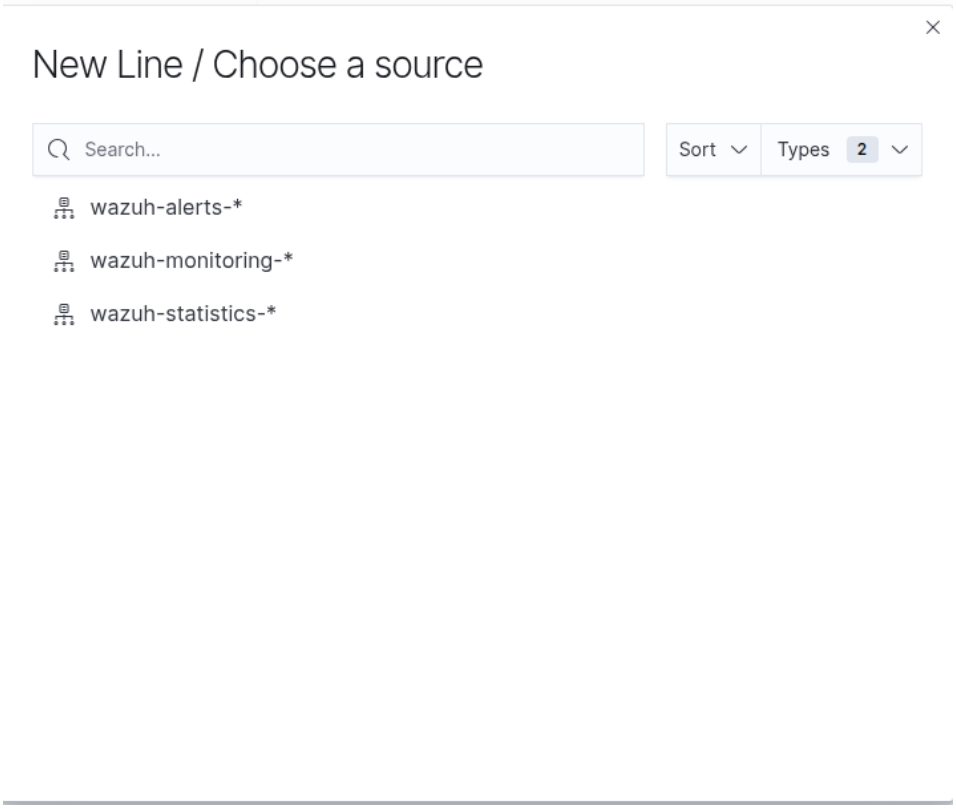



**Figure 3:** *Data source binding step selecting the 'wazuh-alerts-*' index pattern for real-time SIEM alert querying.*

### 2.2 Panel 1: Failed Logins Over Time (Line Visualization)

The first analytical panel monitors brute-force velocity by plotting the count of individual PAM authentication failures over time. The visualization was configured with the following parameters:

- **Visualization Type:** Line Chart (Continuous time-series trend)
- **DQL Search Filter:** rule.id: "5503" (Isolates standard Linux PAM authentication failures)
- **Y-Axis Metric:** Count of matching alert documents
- **X-Axis Bucket:** Date Histogram on @timestamp (30-second intervals)
- **Saved Panel Title:** Failed Logins over time

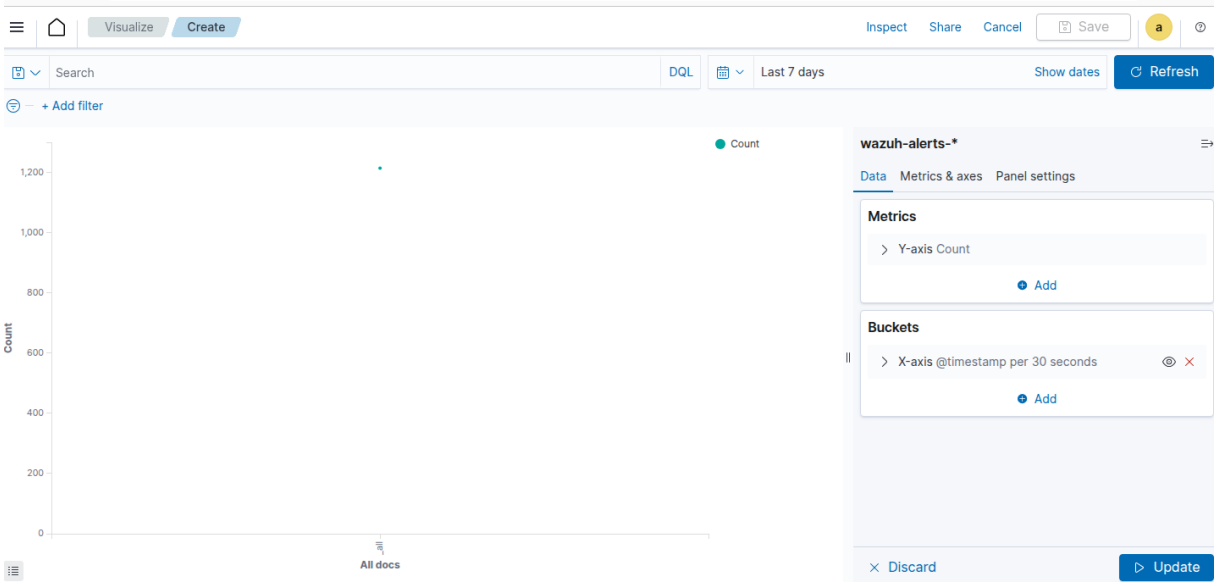



**Figure 4:** *Initial Line visualization canvas bound to 'wazuh-alerts-*' showing default Y-axis Count and X-axis @timestamp buckets before applying the rule filter.*

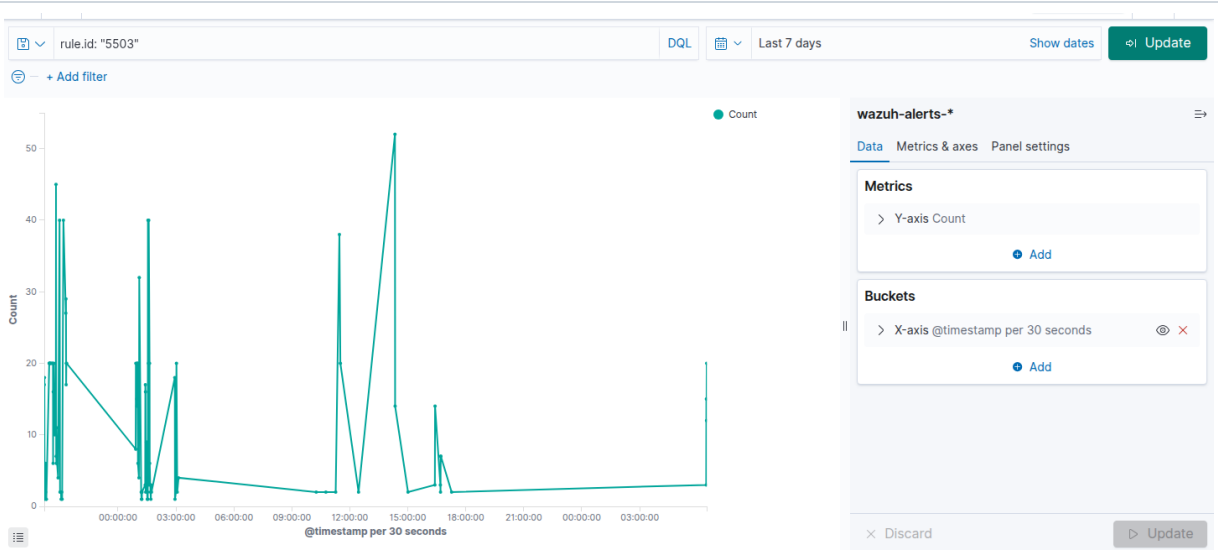



**Figure 5:** *Line visualization filtered with DQL query 'rule.id: "5503"', revealing distinct spikes in authentication failure volume across 30-second intervals.*

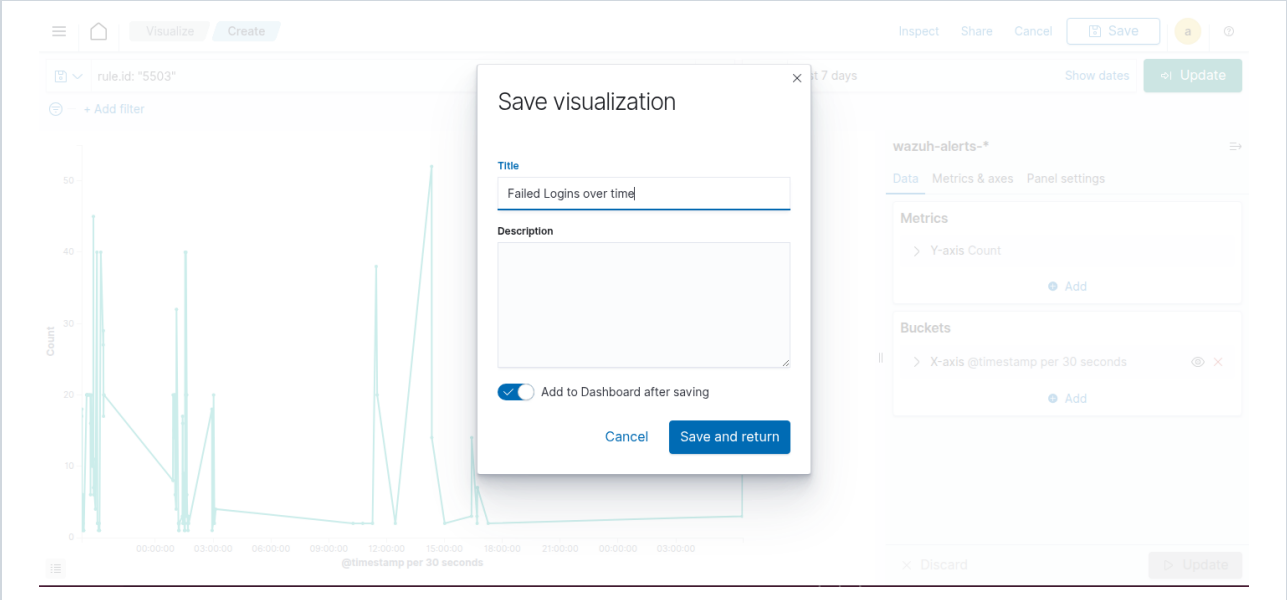



**Figure 6:** *Saving the first visualization as 'Failed Logins over time' with 'Add to Dashboard after saving' enabled.*

## 2.3 Panel 2: Top Security Techniques (MITRE ATT&CK Pie/Donut Chart)

The second panel maps observed alerts across the MITRE ATT&CK framework to highlight prevalent adversarial techniques across the monitored Ubuntu host:

- **Visualization Type:** Pie / Donut Chart
- **Slice Size Metric:** Count of alerts
- **Bucket Aggregation:** Split Slices using Terms aggregation on field 'rule.mitre.technique' ordered descending by Metric: Count
- **Observed Techniques:** Valid Accounts, Sudo and Sudo Caching, Password Guessing, Stored Data Manipulation, and Data Destruction
- **Saved Panel Title:** Top Security Techniques

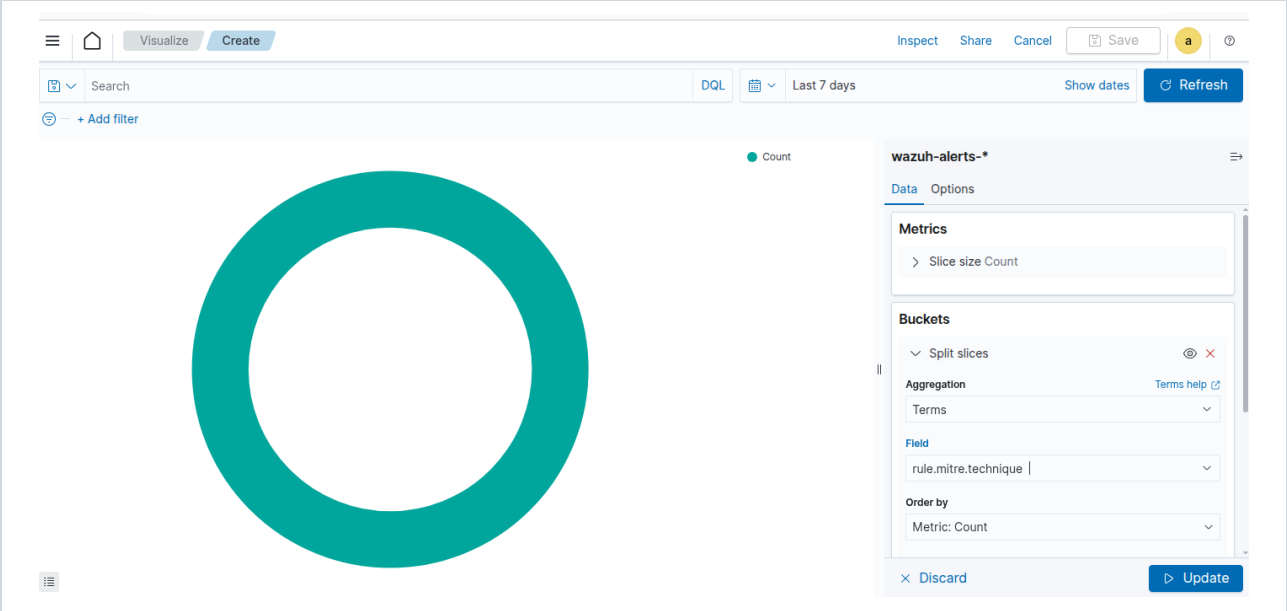



**Figure 7:** *Configuring the Pie/Donut visualization bucket aggregation with Terms on 'rule.mitre.technique' ordered by Metric: Count.*

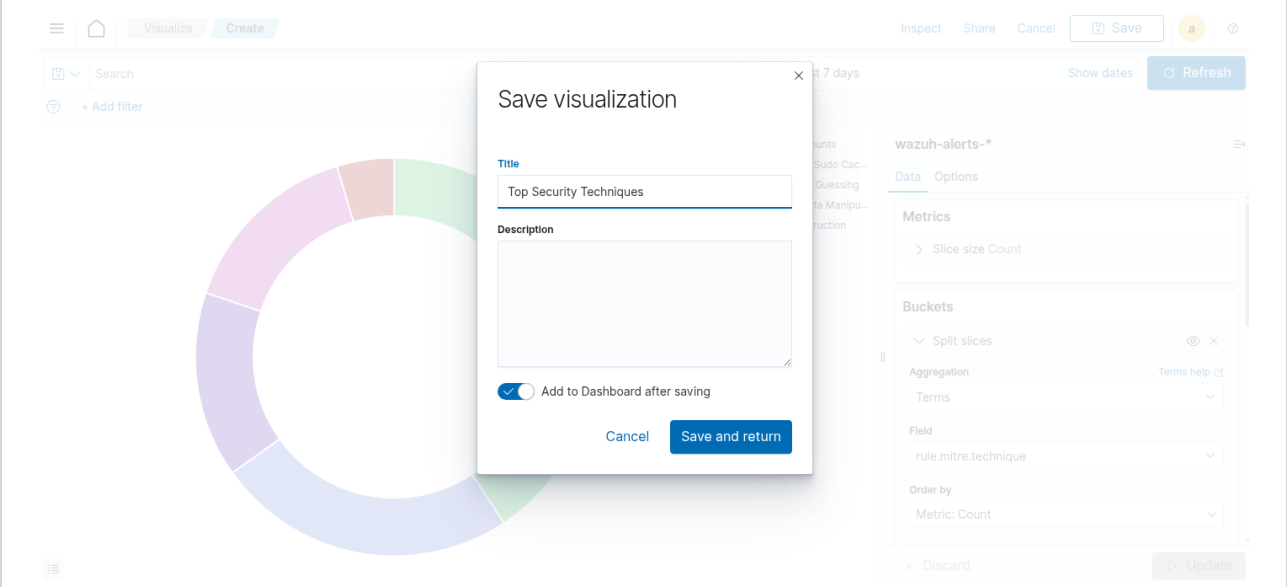



**Figure 8:** *Saving the MITRE ATT&CK breakdown chart as 'Top Security Techniques' and embedding it into the active dashboard.*

## 2.4 Consolidated Two-Panel Dashboard Layout

Both panels were arranged side-by-side in Edit mode and committed to persistent storage using the Save Dashboard dialog. This unified layout correlates raw authentication failure spikes (left) directly with high-level adversary tactic classifications (right).

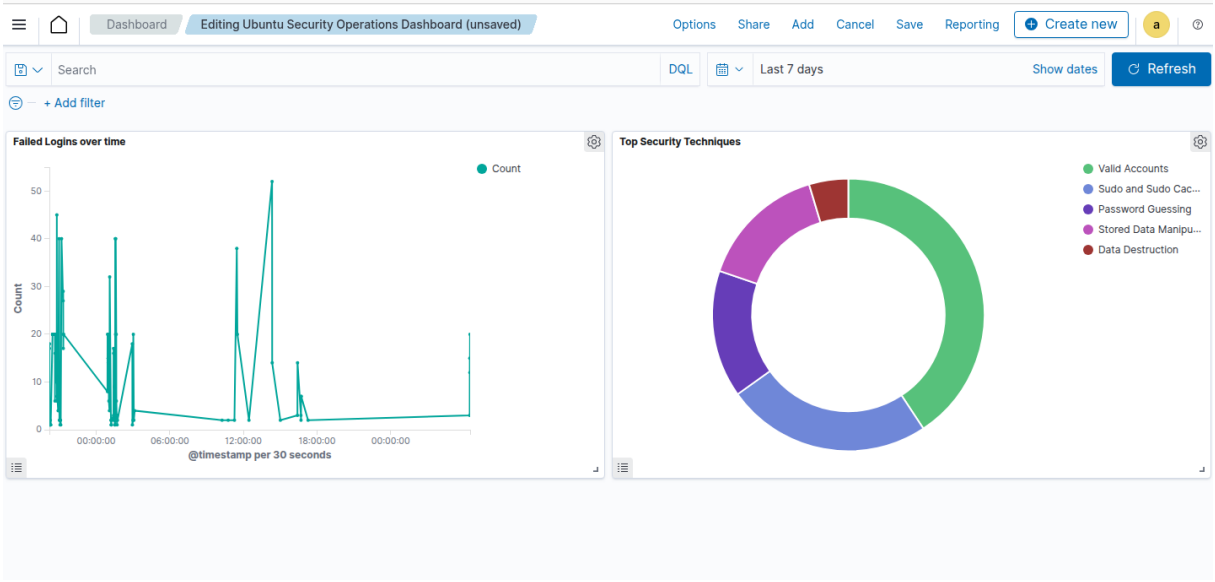



**Figure 9:** *Two-panel layout in Edit mode displaying 'Failed Logins over time' alongside 'Top Security Techniques' prior to final commit.*

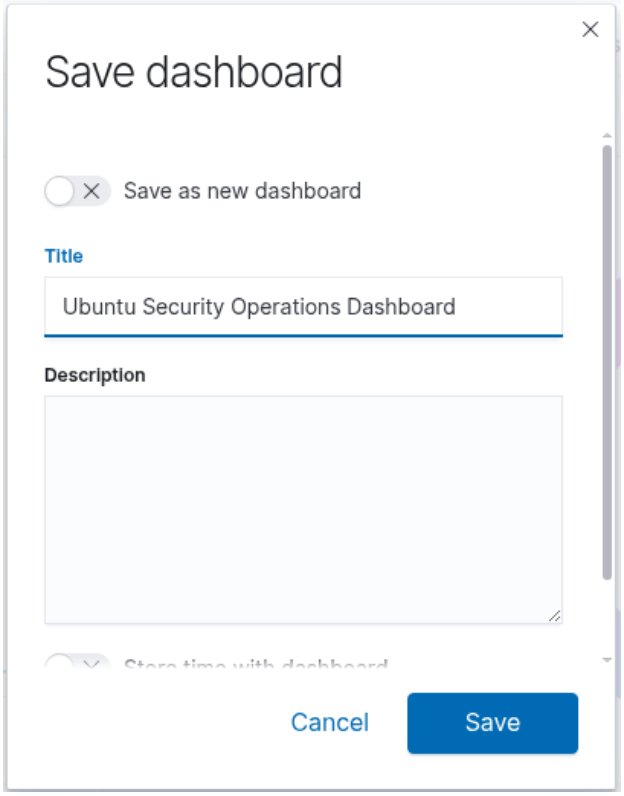



**Figure 10:** *Dashboard persistence modal confirming the saved title 'Ubuntu Security Operations Dashboard'.*

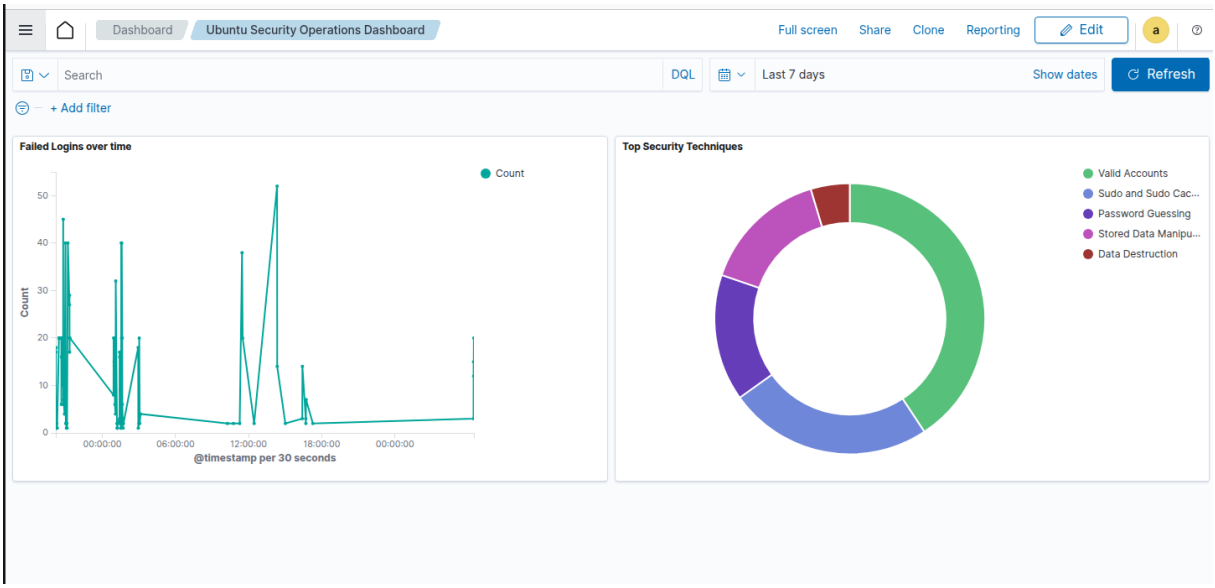



**Figure 11:** *Completed and saved 'Ubuntu Security Operations Dashboard' in production view mode showing live telemetry across both panels.*

## 03 Custom Alert Rule Engineering (Rule 100201)

While Wazuh's default ruleset logs individual failed authentication attempts under Rule 5503 (PAM: User login failed), single failures often represent benign user typos. To detect active password-guessing or brute-force attacks without alert fatigue, a stateful correlation rule was engineered in Wazuh's local rules file (/var/ossec/etc/rules/local_rules.xml) via Management > Rules > Custom rules.

### 3.1 XML Rule Definition

The following XML rule block was authored and saved into local_rules.xml:

```
<group name="linux, authentication, lab_alerts">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100201" level="10" frequency="5" timeframe="600">
    <if_matched_sid>5503</if_matched_sid>
    <description>High frequency failed logins detected: >5 failures in 10 minutes.</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>
</group>
```

### 3.2 Technical Breakdown of Rule Attributes

Each XML element and attribute serves a specific correlation and classification role within the Wazuh analysis daemon (wazuh-analysisd):

| XML Element / Attribute | Configured Value | Functional Security Role |
|---|---|---|
| **<group name="...">** | linux, authentication, lab_alerts | Categorizes the alert under Linux authentication telemetry and custom lab rules for downstream filtering. |
| **rule id** | 100201 | Assigns a unique identifier within Wazuh's custom rule namespace (100000–120000) to prevent overwrite during updates. |
| **level** | 10 | Elevates severity to High (Level 10), distinguishing sustained brute-force activity from routine Level 5 single failures. |
| **frequency & timeframe** | frequency="5" timeframe="600" | Stateful sliding-window counter: triggers only when the parent signature matches 5 or more times within 600 seconds (10 minutes). |
| **<if_matched_sid>** | 5503 | Links this composite rule to Wazuh's built-in PAM authentication failure rule (SID 5503). |
| **<mitre><id>** | T1110 | Enriches the alert with MITRE ATT&CK technique ID T1110 (Brute Force) for automated dashboard mapping. |

### 3.3 XML Editor Deployment & Wazuh Manager Restart

After inserting Rule 100201 into local_rules.xml via the Wazuh Management GUI, the configuration was saved and a manager restart was initiated ('Restarting Manager, please wait.') to compile the updated XML ruleset into active memory.



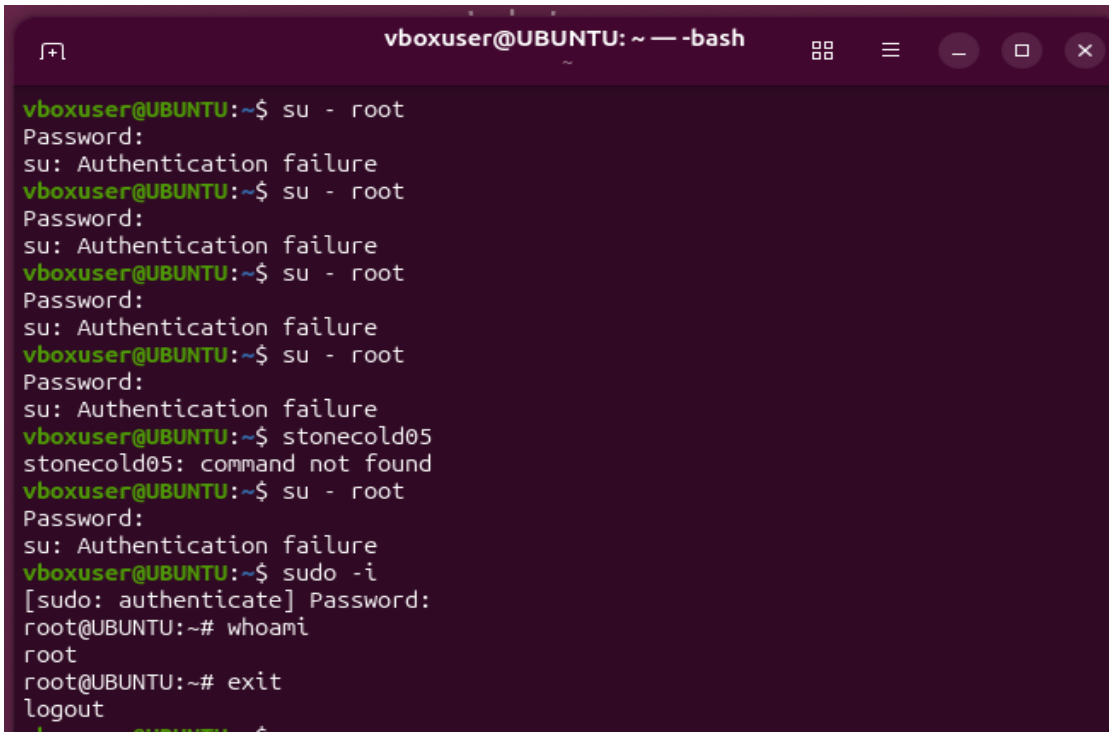

**Figure 12:** *Wazuh Management > Rules editor showing Rule 100201 saved in 'local_rules.xml' while the Wazuh Manager restarts to load the new detection logic.*

## 04 Empirical Trigger Simulation & Alert Verification

To validate that Rule 100201 operates accurately under live attack conditions, a controlled authentication brute-force simulation was conducted on the monitored Ubuntu virtual machine.

### 4.1 Simulation Methodology

- **Brute-Force Pattern Execution:** On the Ubuntu terminal (agent: UBUNTU), rapid consecutive authentication attempts (su - root / user authentication prompts with intentionally invalid credentials) were executed more than 5 times within a 10-minute window.
- **Base Event Generation:** Each failed attempt wrote an authentication failure record to /var/log/auth.log, decoded by Wazuh's PAM decoder and matched against parent Rule 5503.
- **Threshold Correlation:** Upon reaching the configured threshold within the 600-second sliding window, wazuh-analysisd fired composite alert Rule 100201 at Severity Level 10.




### 4.2 Security Events Verification Proof

Navigating to Modules > Security events in the Wazuh web console, setting the time filter to 'Last 15 minutes', and querying DQL filter rule.id: "100201" confirmed the exact alert record captured in Figure 13.

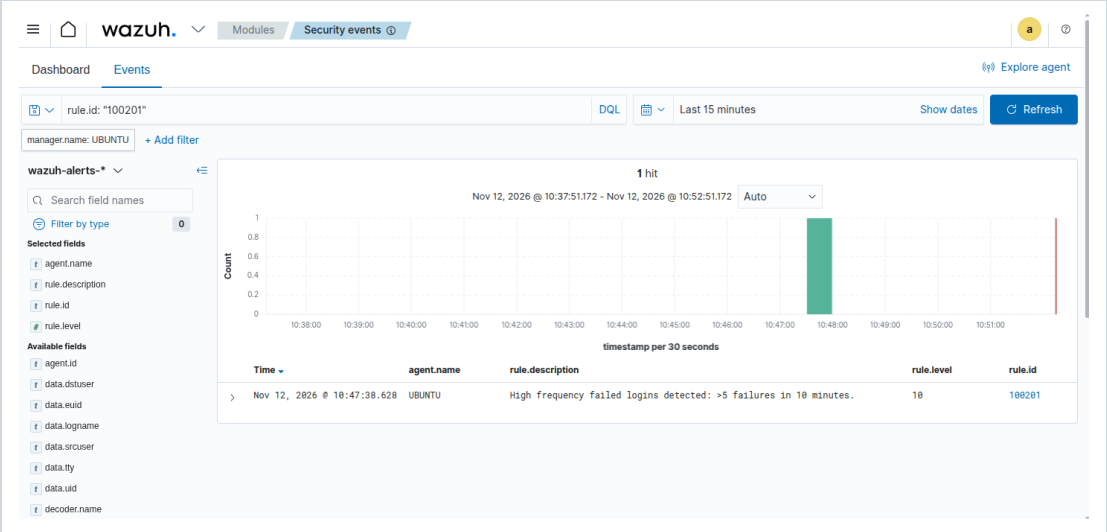



**Figure 13:** *Wazuh Security Events module confirming live trigger of custom Rule 100201 (Level 10) on agent 'UBUNTU' at Nov 12, 2026 @ 10:47:38.628.*

| Event Timestamp | Agent Name | Rule Description | Rule Level | Rule ID |
|---|---|---|---|---|
| Nov 12, 2026 @ 10:47:38.628 | UBUNTU | High frequency failed logins detected: >5 failures in 10 minutes. | **10** | **100201** |

## COMPLETED ROOMS.

